

Handout

Prüfungsvorbereitung

UML:

Anwendungsfalldiagramm (Use-Case_Diagram)

Aktivitätsdiagramm (Activity_Diagram)

Klassendiagramm (Class_Diagram)



Inhalt

1	UML	3
1.1	Erklärung / Definition	3
2	Anwendungsfalldiagramm	3
2.1	Übersicht.....	3
2.1.1	Erklärung / Definition	3
2.2	Syntax.....	4
2.2.1	Anwendungsfall und Akteur	4
2.2.2	Assoziation.....	5
2.2.3	Generalisierung & Spezialisierung	6
2.2.4	Include	7
2.2.5	Extend.....	7
3	Aktivitätsdiagramm	8
3.1	Übersicht.....	8
3.1.1	Erklärung / Definition	8
3.2	Syntax.....	11
3.2.1	Start & Ende	11
3.2.2	Aktion	12
3.2.3	Entscheidung.....	12
3.2.4	Parallelisierung & Synchronisation	13
3.2.5	Partitionen	13
4	Klassendiagramm	14
4.1	Übersicht.....	14
4.1.1	Erklärung / Definition	14
4.2	Syntax.....	15
4.2.1	Die Klasse	15
4.2.2	Das Interface	17
4.2.3	Das Enum	17
4.3	Beziehungen	18



4.3.1	Assoziation.....	18
4.3.2	Aggregation.....	19
4.3.3	Komposition	19
4.3.4	Vererbung.....	20
4.3.5	Implementierung.....	20
4.3.6	Weitere	21

1 UML

1.1 Erklärung / Definition

Die Unified Modeling Language (UML) ist eine standardisierte grafische Sprache, die zur Modellierung und Visualisierung von Software-Systemen verwendet wird. Sie ermöglicht es Entwicklern, komplexe Systeme zu strukturieren und zu verstehen, indem sie verschiedene Diagrammtypen wie Klassendiagramme, Sequenzdiagramme und Use-Case-Diagramme bereitstellt.

2 Anwendungsfalldiagramm

2.1 Übersicht

2.1.1 Erklärung / Definition

Ein UML Use-Case-Diagramm, auch Anwendungsfalldiagramm genannt, ist ein Verhaltensdiagramm, das die erwarteten Funktionen eines Systems oder einer Software aus der Perspektive eines Akteurs darstellt. Dieser Akteur kann sowohl ein menschlicher Benutzer als auch ein externes System sein. Es ist wichtig zu beachten, dass ein Use-Case-Diagramm nicht die genaue Reihenfolge der Abläufe darstellt, dafür werden andere UML-Diagramme wie Aktivitäts- oder Sequenzdiagramme verwendet. Im Wesentlichen bietet ein Use-Case-Diagramm eine abstrakte Übersicht über die Funktionen und Dienste, die ein System für einen Anwender bereitstellt.

Text

Beispiel 1:

Kunde kann Geld auszahlen lassen, dabei muss Kunde immer die PIN eingeben. Ist die PIN falsch, wird der Vorgang abgebrochen. Der Vorgang kann zu jeder Zeit auch durch Kunde abgebrochen werden. Kunde kann Geld einzahlen.

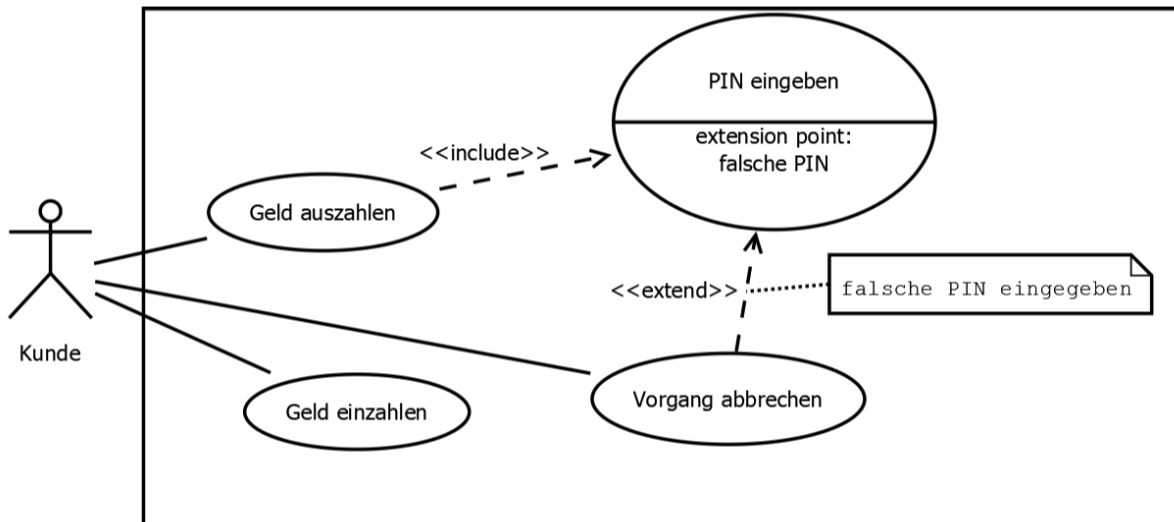
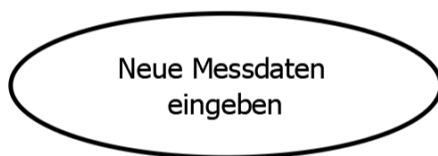


Abb.2.1 Darstellung Use-Case-Diagramm Kunde-Geldautomat

2.2 Syntax

2.2.1 Anwendungsfall und Akteur



Ein Anwendungsfalldiagramm besteht aus einem oder mehreren Anwendungsfällen. Anwendungsfälle beschreiben die Funktionen, die ein Akteur ausführen/auslösen/durchführen kann

Akteure sind Personen oder Systeme, die mit der Anwendung interagieren. Hierbei werden nicht direkt Menschen bezeichnet, sondern Rollen oder Personengruppen definiert.

Zum Beispiel „Buchhaltung“, oder „Benutzer“, „Admin“

Akteure stehen immer außerhalb des zu modellierenden Systems!

2.2.2 Assoziation

Akteure sind über Assoziationen mit den Anwendungsfällen verbunden.

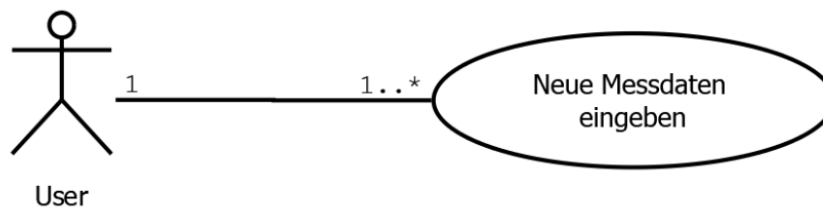


Abb.2.2 Darstellung Assoziation User zu Anwendungsfall „Neue Messdaten eingeben“

Diese Assoziationen können über Multiplizitäten verfügen (ein User kann 1 oder mehrere neue Messdaten eingeben, aber ein Set von Messdaten wird nur von einem User eingegeben)

Sie können auch beschriftet werden und Pfeilspitzen haben. Die Pfeilspitzen geben hier die möglichen Kommunikationsrichtungen an. (Kunde und Mitarbeiter können in beide Richtungen miteinander kommunizieren)

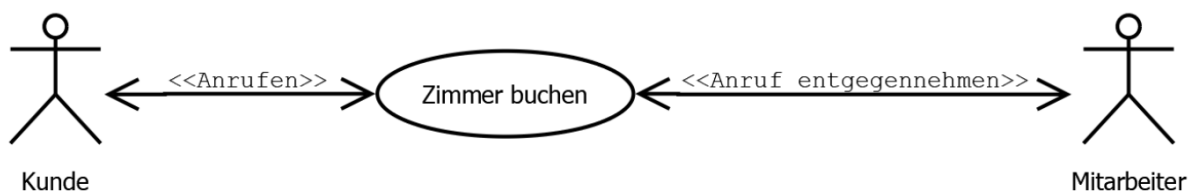


Abb.2.3 Darstellung zweier Akteure mit einer Assoziation zum Anwendungsfall „Zimmer buchen“

Ist mehr als ein Akteur an einem Anwendungsfall beteiligt, müssen alle beteiligten Akteure aktiv werden, um den Anwendungsfall auszuführen. (Ein Kunde kann nicht ohne Mitarbeiter ein Zimmer buchen)

2.2.3 Generalisierung & Spezialisierung

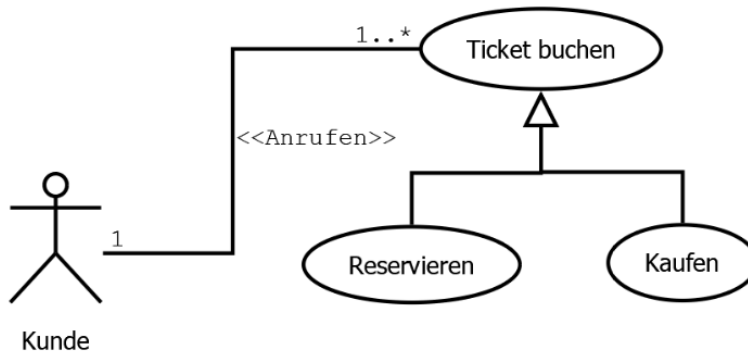


Abb.2.4 Darstellung Generalisierung & Spezialisierung von Anwendungsfällen

Anwendungsfälle können generalisiert und spezialisiert sein. Dies funktioniert ähnlich wie die Vererbung bei Klassen. Hier erben „Reservieren“ und „Kaufen“ alle Eigenschaften und Assoziationen von „Ticket buchen“.

Da „Kunde“ den Anwendungsfall „Ticket buchen“ ausführen kann, kann „Kunde“ auch „Reservieren“ und „Kaufen“ ausführen.

Auch Akteure können generalisiert und spezialisiert sein.

Hier erbt „Registrierter Kunde“ alle Eigenschaften und Assoziationen von „Gast-Kunde“.

Da „Gast-Kunde“ den Anwendungsfall „Reservieren“ ausführen kann, kann „Registrierter Kunde“ auch „Reservieren“ ausführen. Zusätzlich kann „Registrierter Kunde“ aber auch „Kaufen“ ausführen, was „Gast-Kunde“ nicht kann.

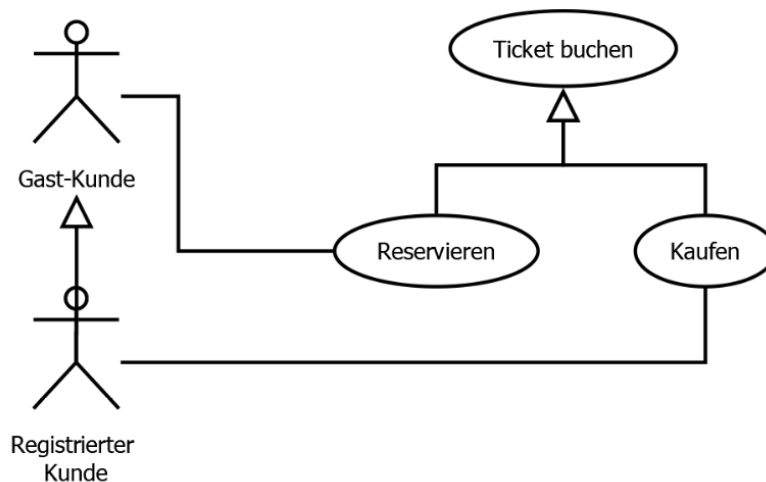


Abb.2.5 Darstellung Generalisierung und Spezialisierung von Anwendungsfällen und Akteuren

2.2.4 Include

Ein Anwendungsfall kann einen anderen Anwendungsfall einbinden. Dies wird über „include“ gekennzeichnet. Der eingebundene Anwendungsfall wird immer und automatisch zusammen mit dem ursprünglichen Anwendungsfall ausgeführt. (Werden neue Messdaten eingegeben, werden die Daten automatisch analysiert)

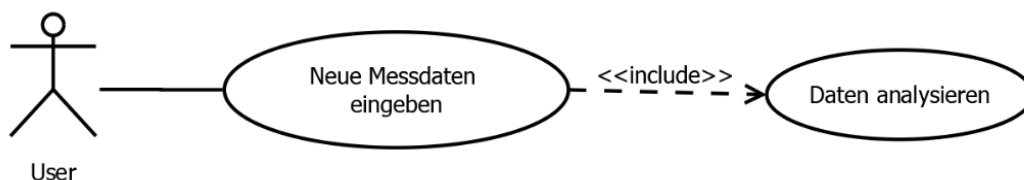


Abb.2.6 Darstellung eines Einbinden eines Anwendungsfall durch include

Dies ist praktisch, wenn zum Beispiel noch weitere Akteure an dem Inkludierten Use-Case beteiligt sind oder der Use-Case für Generalisierung/Spezialisierung verwendet werden soll. Include kann aber auch einfach nur dazu verwendet werden, Zusammenhänge zu modellieren und die Anwendung genauer zu beschreiben.

Wichtig: Auf die Pfeilrichtung achten! Der Pfeil zeigt auf den eingebundenen Anwendungsfall.

2.2.5 Extend

Ein Anwendungsfall kann einen anderen Anwendungsfall erweitern. Dies wird über „extend“ gekennzeichnet. Der erweiternde Anwendungsfall wird nur unter einer bestimmten Bedingung mit dem erweiterten Anwendungsfall ausgeführt.

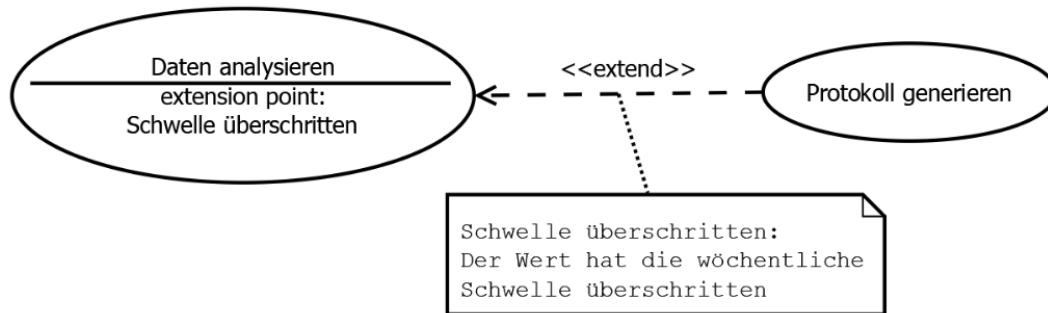


Abb.2.7 Darstellung einer Erweiterung eines Anwendungsfalls durch extend

Die Bedingung wird als „Extension Point“ bezeichnet und kann über Notizen detaillierter beschrieben werden.

Wichtig: Auf die Pfeilrichtung achten! Der Pfeil zeigt auf den erweiterten Anwendungsfall.

3 Aktivitätsdiagramm

3.1 Übersicht

3.1.1 Erklärung / Definition

Ein UML-Aktivitätsdiagramm ist ein nützliches Werkzeug, das den Ablauf von Aktivitäten in einem System darstellt. Es wird verwendet, um spezifische Anwendungsfälle detailliert zu visualisieren. Hier sind einige seiner Anwendungen:

Software-Entwicklung: Es unterstützt verschiedene Phasen und Zwecke der Software-Entwicklung.

Geschäftsprozesse: Es hilft bei der Darstellung und Untersuchung des Ablaufs von Geschäftsprozessen.

Anforderungsmodellierung: Es wird zur Modellierung von Geschäftsanforderungen verwendet.

Systemfunktionalität: Es bietet eine Übersicht über die Funktionen eines Systems.

Anwendungsfallanalyse: Es wird zur Analyse von Anwendungsfällen verwendet.

Es ist wichtig zu beachten, dass Aktivitätsdiagramme sowohl sequenzielle als auch parallele Aktivitäten darstellen können. Sie werden in der Prozessmodellierung eingesetzt und zeigen die dynamischen Aspekte eines Systems. Sie ähneln in vielerlei Hinsicht einem Programmablaufplan.

Beispiel 1:

Das folgende Aktivitätsdiagramm stellt den Prozess der Geldabhebung an einem Geldautomaten dar. Hier sind die Schritte:

- Der Kunde beginnt mit dem Einführen einer Bankkarte.
- Es gibt eine Auswahlaktion, bei der Optionen wie Einzahlung, Abhebung und Kontostandabfrage zur Verfügung stehen.
- Wenn “Abheben” ausgewählt ist, muss der Kunde den Betrag eingeben und die PIN eingeben.
- Wenn die PIN korrekt ist und Geld vorhanden ist, wird das Geld ausgezahlt; wenn nicht, wird eine Meldung ausgegeben.
- Bei einer falschen PIN-Eingabe gibt es Optionen für eine neue Eingabe oder einen Abbruch des Vorgangs.
- Nach drei fehlgeschlagenen Versuchen wird die Karte gesperrt.

Dieses Diagramm ist ein gutes Beispiel für die Darstellung von Entscheidungsprozessen und Aktionen in einem System. Es hilft dabei, die verschiedenen Szenarien und Ergebnisse zu verstehen, die aufgrund von Benutzereingaben auftreten können. Es ist ein nützliches Werkzeug für die Systemanalyse und -design.

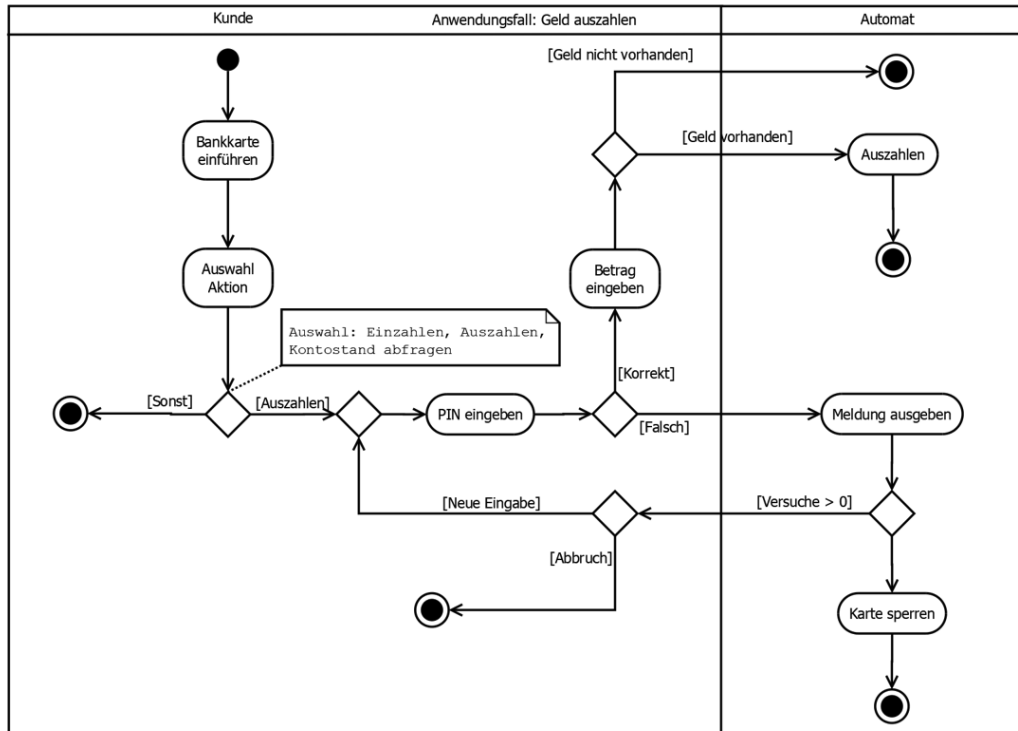


Abb. 3.1 Darstellung des Prozess der Geldabhebung am Geldautomaten

Beispiel 2:

Das folgende Aktivitätsdiagramm stellt den Prozess des Einzahlens von Geld an einem Geldautomaten dar. Hier sind die Schritte:

- Der Kunde beginnt mit dem Einführen einer Bankkarte.
- Es gibt eine Auswahlaktion, bei der Optionen wie Einzahlung, Abhebung und Kontostandabfrage zur Verfügung stehen.
- Wenn "Einzahlen" ausgewählt ist, geht der Prozess weiter zum "Geld in Automat legen" und dann zur Bestätigung, bevor er endet.
- Wenn eine andere Option ausgewählt wird, endet der Prozess.

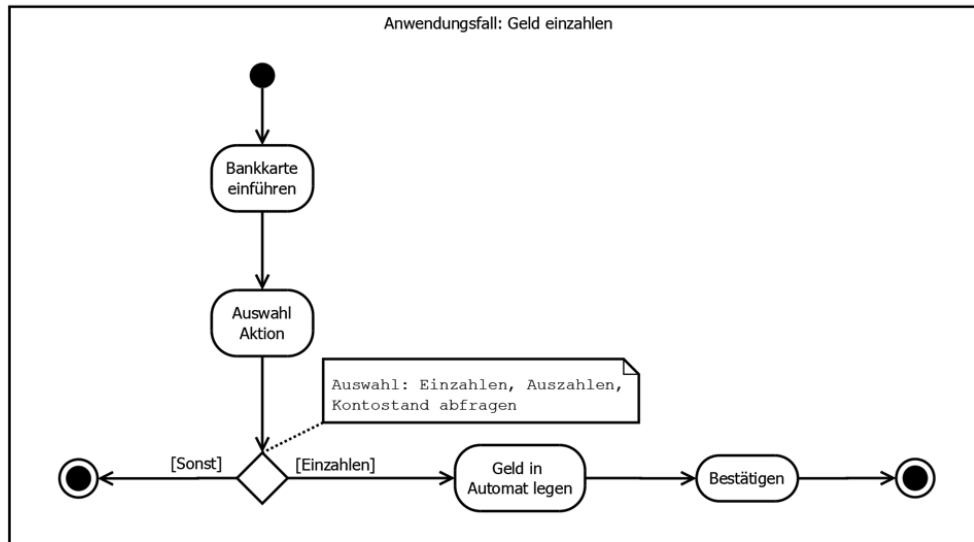
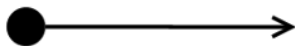


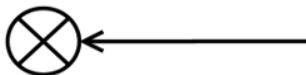
Abb. 3.2 Darstellung des Prozess des Einzahlens von Geld über einen Gelautomaten

3.2 Syntax

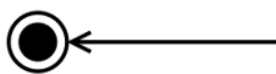
3.2.1 Start & Ende



Jedes Aktivitätsdiagramm beginnt mit dem Startknoten

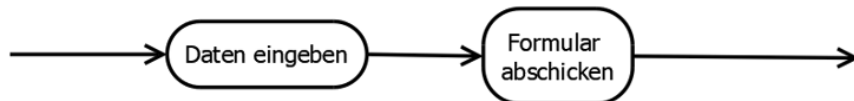


Das Ende eines Ablaufpfades kann durch das Pfadende markiert werden
Pfadende beendet nur einen Pfad, nicht die gesamte Aktivität!



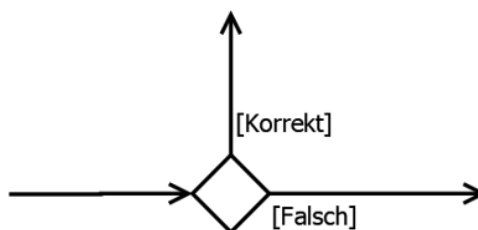
Jedes Aktivitätsdiagramm hat mindestens einen Endpunkt, es kann aber auch mehrere haben.

3.2.2 Aktion



Ein Aktivitätsdiagramm besteht aus einer oder mehreren Aktionen, die über Pfeile miteinander verbunden sind. Aktionen sind das, was ein Benutzer während der Aktivität ausführt.

3.2.3 Entscheidung



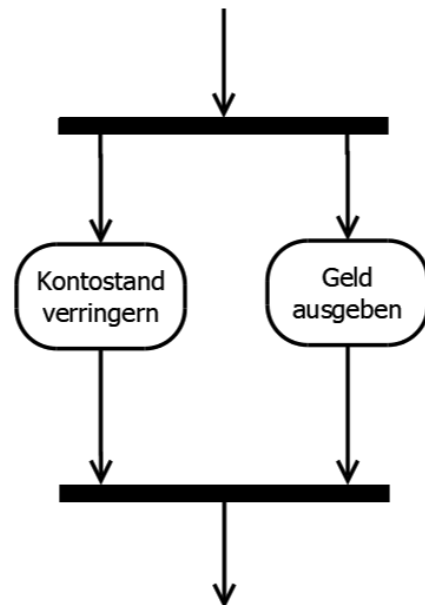
Hat eine Aktion mehrere mögliche Ergebnisse (z.B. war die eingegebene PIN korrekt oder nicht?), wird dies über Entscheidungen modelliert. Entscheidungen werden mit Bedingungen beschriftet.

In eine Entscheidung können mehrere Pfeile eingehen und es können mehrere Pfeile aus ihr heraus gehen. Eine Entscheidung teilt den Aktivitätsfluss auf mehrere unabhängige Pfade auf. Treffen sich mehrere Pfade wieder, so wird dies ebenfalls mit der Raute dargestellt.

3.2.4 Parallelisierung & Synchronisation

Wird ein Aktionsfluss in mehrere gleichzeitig stattfindende Pfade aufgeteilt, findet eine Parallelisierung statt. Aktionen werden parallel, relativ gleichzeitig ausgeführt.

Sollen parallele Pfade wieder zusammengeführt werden, findet eine Synchronisation statt. Erst wenn alle parallelen Aktionen ausgeführt wurden, können die Pfade wieder synchronisiert werden.



3.2.5 Partitionen

Kunde	Mitarbeiter

An Aktivitäten sind häufig mehrere Akteure beteiligt. Um dies darzustellen wird das Diagramm in Partitionen (oft auch Schwimmbahn genannt) unterteilt. Aktionen, die ein Akteur ausführt, wird in dessen Partition platziert.

4 Klassendiagramm

4.1 Übersicht

4.1.1 Erklärung / Definition

Ein UML-Klassendiagramm ist ein visuelles Werkzeug, das in der Unified Modeling Language (UML) verwendet wird. Es stellt Klassen, Schnittstellen und ihre Beziehungen dar, um ein System zu beschreiben. Es visualisiert die verschiedenen Objekttypen und ihre statischen Beziehungen innerhalb eines Systems.

In einem Klassendiagramm werden Klassen durch Rechtecke dargestellt, die den Namen der Klasse, ihre Attribute und Operationen enthalten können. Diese drei Elemente sind durch horizontale Linien getrennt. Das Diagramm hilft dabei, die Struktur eines Systems zu analysieren und zu verstehen.

Kurz gesagt:

Das UML-Klassendiagramm wird verwendet, um die Klassen in einem Programm und deren Beziehungen zueinander zu dokumentieren und Entwicklern eine Übersicht zu geben.

Beispiel 1:

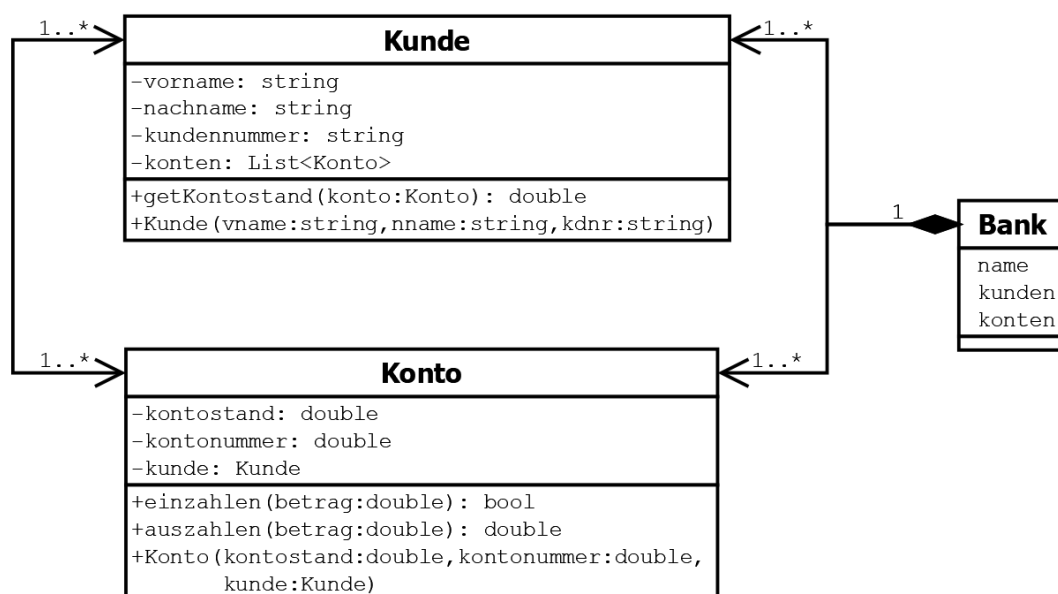


Abb. 4.1 Darstellung -Klassendiagramm (Bank-Kunde-Konto)

Die Darstellung zeigt ein UML-Klassendiagramm, das die Beziehungen zwischen einer Bank, ihren Kunden und deren Konten repräsentiert. Es besteht aus drei Klassen: "Kunde", "Konto" und "Bank".

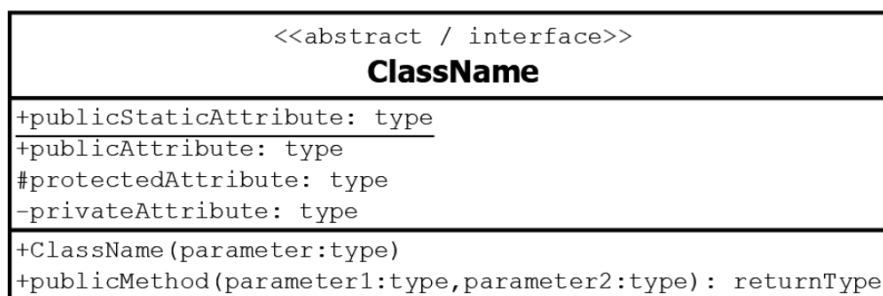
- Die Klasse *Kunde* hat Attribute wie Vorname, Nachname, Kundennummer und eine Liste von Konten. Sie hat auch Methoden, um den Kontostand abzurufen und einen neuen Kunden zu erstellen.
- Die Klasse *Konto* enthält Attribute wie Kontostand und Kontonummer. Sie hat Methoden zum Ein- und Auszahlen von Geld sowie einen Konstruktor zum Erstellen eines Kontos.
- Die Klasse *Bank* ist mit beiden, "Kunde" und "Konto", verbunden, was darauf hinweist, dass sie Informationen über Kunden und deren Konten enthält.

Die Beziehungen sind wie folgt:

- Eine Bank kann mehrere Kunden haben (1...*).
- Ein Kunde kann mehrere Konten haben (1...*).
- Ein Konto ist mit einem Kunden verbunden, könnte aber potenziell über den Kunden mit mehreren Banken verknüpft sein.

4.2 Syntax

4.2.1 Die Klasse



Die Klasse ist in 3 Bereiche eingeteilt:

Klassenname: Dieser steht im oberen Bereich des Rechtecks und beginnt immer mit einem Großbuchstaben, optional dazu der Stereotyp

Attribute: Sie repräsentieren die Merkmale der Objekte und werden im mittleren Bereich des Rechtecks aufgelistet.

Operationen: Sie stellen das Verhalten der Klasse dar und listen die Methoden auf, die die Objekte ausführen können. Sie werden im unteren Bereich des Rechtecks aufgeführt.

Attribute und Methoden werden zusammen als die „Member der Klasse“ bezeichnet

Stereotyp:

- `<<abstract>>` bei abstrakten Klassen
- `<<interface>>` bei Interfaces
- `<<struct>>` bei Strukturen
- `<<enum>>` für Enumeratoren

Syntax für Attribute:

Sichtbarkeit Attributname : Typ {Eigenschaften}

Syntax für Methoden:

Sichtbarkeit Methodenname(Parameterliste) : Rückgabotyp {Eigenschaften}

Parameter:

Parametername : Typ

Sichtbarkeit:

+ public

protected

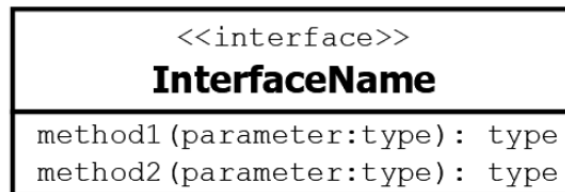
- private

~ package

Eigenschaften:

{static, final, abstract, ...}

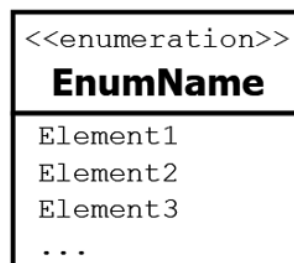
4.2.2 Das Interface



Im Unterschied zu Klassen, werden Interfaces¹ häufig nur mit zwei Bereichen dargestellt. Kopf und Methoden, drei Bereiche sind aber nicht unüblich (gerade bei IHK-Aufgaben kommt das vor)

Wie bereits erwähnt, werden Interfaces mit <<interface>> ausgezeichnet. Interface-Member haben keine Zugriffsmodifizierer/Sichtbarkeiten (manchmal sind sie aber auch einfach public)

4.2.3 Das Enum



In einem UML-Klassendiagramm wird ein Enum² (Enumeration) als Rechteck dargestellt, das seine möglichen Werte auflistet. Der Name des Enums steht oben, während die einzelnen Werte darunter aufgelistet sind. Um es von anderen Elementen zu unterscheiden, wird das Rechteck mit dem Stereotyp «enumeration» gekennzeichnet. Ein Enum kann auch mit einer

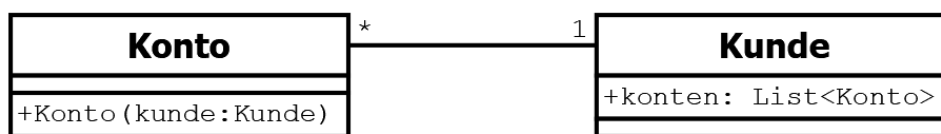
¹ In der Programmierung ist ein Interface (oder Schnittstelle) ein Konstrukt, das die Struktur des Codes definiert. Es legt die Methoden fest, die eine Klasse implementieren muss, und dient als Standard für die Kommunikation zwischen zwei Parteien. Wenn eine Klasse ein Interface implementiert, muss sie alle Methoden mit den entsprechenden Signaturen bereitstellen. Interfaces ermöglichen die Austauschbarkeit von Instanzen unabhängig von ihrem Typ und trennen den Kontrakt von der Implementierung. Es ist zu beachten, dass die Implementierung von Interfaces in verschiedenen Programmiersprachen variiert.

² Ein Enum (Enumeration) ein spezieller Datentyp, der eine Reihe von benannten Konstanten definiert. Enums verbessern die Lesbarkeit und Wartbarkeit des Codes, da jeder Konstante ein Name gegeben wird. Es ist zu beachten, dass die Implementierung von Enums in Java von anderen Programmiersprachen abweichen kann.

Klasse verbunden sein, um anzuzeigen, dass ein Attribut, eine Operation oder ein Parameter den Enum als Typ verwendet.

4.3 Beziehungen

4.3.1 Assoziation



Sobald eine Klasse eine andere Klasse kennt, entweder über Attribute oder Methodenparameter, liegt eine Assoziation vor. Die beiden Klassen stehen in einer Beziehung zueinander.

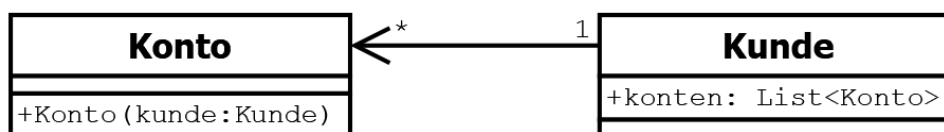
Ähnlich wie bei Datenbank-Diagrammen, werden hier Multiplizitäten (Bei DB-Kardinalität genannt) angegeben.

* = beliebig viele

1..* = mindestens 1

2..5 = mindestens 2, höchstens 5

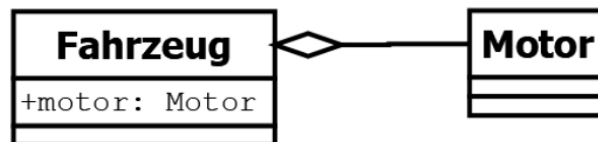
1 = genau 1



Assoziationen können auch Pfeilspitzen haben. Hierbei handelt es sich dann um eine gerichtete Assoziation. Der Pfeil zeigt auf die Klasse, auf die navigiert werden kann. Habe ich ein Objekt von Kunde, kann ich über Kunde nun auf die Konto-Objekte navigieren und z.B. von Konto Methoden aufrufen.

Assoziationen können auch beschriftet werden und gelegentlich haben sie auch Pfeile, die die Leserichtung angeben.

4.3.2 Aggregation



Ein Spezialfall von Assoziation ist die Aggregation.

Sobald eine Klasse aus Objekten einer anderen Klasse besteht, ist es eine Aggregation. Hier sprechen wir von „hat“, „ist Teil von“ oder „besteht aus“. Das Fahrzeug hat einen Motor. Damit liegt eine Aggregation vor. Bei der Aggregation ist wichtig, dass das Ganze auch tatsächlich Teile haben muss. Motor darf hier also nicht Null sein.

Die Navigierbarkeit ist hierbei in der Regel vom Ganzen auf die Teile (also hier von Fahrzeug auf Motor).

4.3.3 Komposition

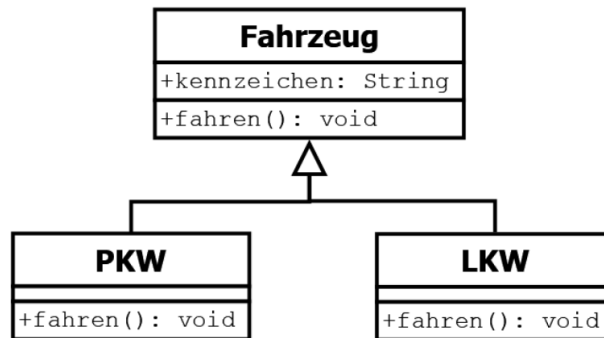


Ein weiterer Spezialfall ist die Komposition.

Es gelten dieselben Bedingungen wie bei einer Aggregation, der Unterschied liegt aber darin, dass bei einer Komposition eine existentielle Abhängigkeit vorliegt. Das eine kann nicht ohne das andere existieren.

Ein Raum kann nicht ohne ein passendes Gebäude existieren. Werden alle Teile gelöscht, wird auch das Ganze gelöscht. Wird das Ganze gelöscht, verschwinden auch alle Teile.

4.3.4 Vererbung

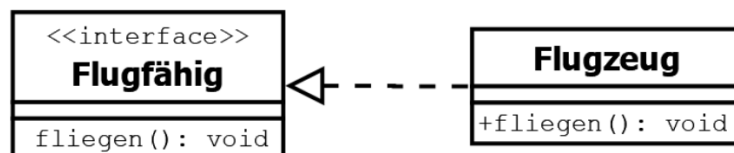


Klassen können voneinander erben. Es wird auch von Generalisierung/Spezialisierung gesprochen. Hier liegt eine „ist“-Beziehung vor. Ein PKW IST EIN Fahrzeug. Ein LKW IST EIN Fahrzeug.

Wichtig: Nicht jedes Fahrzeug ist auch automatisch ein PKW oder LKW!

Member, außer Konstruktoren, werden von der Eltern-Klasse (auch Basis-Klasse oder Super-Klasse genannt) an die Kind-Klassen (auch Sub-Klasse genannt) vererbt. Im Klassendiagramm werden geerbte Member nicht nochmal wiederholt, außer sie werden überschrieben, zum Beispiel wenn der Member der Basis-Klasse abstract ist oder man möchte ein neues Verhalten implementieren.

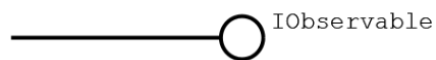
4.3.5 Implementierung



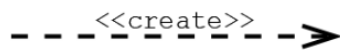
Interfaces werden nicht geerbt, sondern implementiert. Es wird gelegentlich auch von Realisierung gesprochen. Wie bei der Vererbung handelt es sich hier um eine „ist“-Beziehung. Das Flugzeug IST Flugfähig, da es das Interface Flugfähig implementiert.

Im Klassendiagramm müssen alle Nicht-Standard-Schnittstellenmember in der implementierenden Klasse wiederholt werden.

4.3.6 Weitere



Eine Klasse implementiert eine externe, nicht genauer spezifizierte Schnittstelle.



Abhängigkeit: Die Klasse wird nicht als Attribut (Assoziation) verwendet, sondern als Objekte zum Beispiel durch Methoden erzeugt.